

MACE: A Multi-core Agentic Co-design Engine for Autonomous OpenPiton Bring-Up

Rana Umar Nadeem

CHIA A3 Hackathon Submission, September 2026

Abstract—MACE is an agentic AI system that builds and verifies multicore hardware on its own. It runs on Google and NVIDIA’s CHIA framework. Phase 1 adds `chia_openpiton`, a CHIA adapter for OpenPiton’s `configure`, `build`, `simulate`, `run`, and `collect` operations. We validated it with 208 real tests. Phase 2 adds the MACE loop: a planner breaks a hardware objective into tasks, agents run tasks in parallel, and a deterministic integrator gates every change against real Verilator simulation before accepting it. A failure-analysis agent diagnoses any failure and the planner replans around it.

We tested MACE against two baselines, manual mesh scaling and a one-shot LLM prompt, on all three supported cores, using real hardware throughout. No result in this paper is simulated. MACE found and fixed a real RTL bug while adding a third core, PicoRV32. We found and fixed a real CHIA dispatch bug. The one-shot baseline failed on all three cores in three different ways; MACE passed one and correctly stopped within its retry budget on the other two instead of reporting a false pass. MACE also passed two mesh sizes, 2x2 and 4x4, that manual scaling could not reach.

1. Introduction

Agentic coding systems are usually tested on small, isolated tasks: one file, one test, a toy problem. Real hardware construction does not look like that. A real multicore system means a large RTL codebase (OpenPiton spans hundreds of thousands of lines across network-on-chip, cache-coherence, tile, and platform logic), a toolchain with real failure modes (version skew, cross-compilation bugs, checkout-specific issues), and verification that costs minutes to hours per attempt, not milliseconds.

MACE targets this harder, more realistic setting. Given a hardware objective, such as “bring up a 2x2 mesh” or “verify workload W passes,” MACE plans, implements, builds, simulates, diagnoses failures against real Verilator output, and iterates. Every claim of success must pass an actual RTL simulation. MACE never accepts an agent’s self-report.

1.1. Features

MACE contributes four things. First, `chia_openpiton` (§3): a CHIA adapter for OpenPiton with a full `configure/build/run/collect` interface, built the same way as CHIA’s

`gem5` and `ESP` adapters, so it can be upstreamed directly. Second, the MACE loop (§4): planning, parallel task execution, deterministic integration, and LLM-driven failure triage, tested end to end on real hardware with real cost, time, and task metrics. Third, we added PicoRV32 as a third core (§5) and found a real, previously unknown bug in its OpenPiton integration; nobody, including OpenPiton’s own continuous integration, had caught it, because nobody had run it under a simulator before. Fourth, a production interface with two new capabilities (§6): a real interactive CLI, a second verification axis (real Verilator code coverage), and a second working LLM backend, none of which the original proposal required.

2. Background

CHIA provides the agent, tool, and dispatch layer this project builds on: colocated resource-pinned worker nodes, an LLM abstraction layer spanning multiple backends, and Ray-based remote dispatch and caching. OpenPiton is Princeton’s open-source, silicon-proven manycore research framework. Tiles connect through a 2D-mesh network-on-chip and speak a proprietary MESI-coherence protocol (L15, descended from OpenSPARC T1’s PCX/CPX bus) to reach the NoC. OpenPiton supports three tile cores: OpenSPARC T1 (`sparc`), Ariane/CVA6 (`ariane`, RV64GC, coherent), and PicoRV32 (`pico`, RV32I, non-coherent). Before this project, CHIA had no OpenPiton adapter.

3. Phase 1: The OpenPiton Adapter

Design. `OpenPitonWorkspaceNode` is a CHIA `ColocatedNode`. It exposes `configure`, `build`, `run`, `regress`, `collect`, and `clean` as resource-pinned remote functions. OpenPiton’s own template preprocessor writes generated `.tmp.v` files directly into the source tree on every build, so concurrent builds cannot share a checkout. The adapter reserves one `openpiton` resource unit per checkout and gives each worker its own checkout instead of locking a shared one. Pure parsers read build and run verdicts directly from OpenPiton’s own simulation transcripts (`sim.log`, `status.log`), using the same strings OpenPiton’s own RTL monitors emit. A verdict never comes from an agent’s own claim.

Real hardening. Building real hardware surfaced four environment bugs. We fixed all four in one idempotent patch

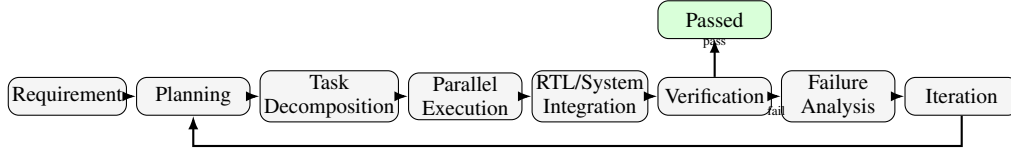


Figure 1. The MACE pipeline. Verification gates every integrated change against real Verilator simulation, never a self-report. Every gate workload passing ends the run. Any failure triggers failure analysis and a replan, not a bare retry.

script: binutils 2.38+ splits `zicsr/zifencei` out of base RV64I and breaks the boot ROM’s own assembly; GCC 15’s C23 default breaks a legacy two-argument function call; broken git symlinks on a Windows-mounted checkout turn into literal path-text files; and 66 files have CRLF-terminated shebang lines. None of these bugs are specific to MACE. Any real deployment of decades-old RTL hits them.

Build-result caching. `build()` skips the simulator entirely when a matching successful-build marker already exists on disk. We measured a real rebuild of an identical configuration: 104s to 0.00s. This gives “an agent iteration that only changes the test should never rebuild” with no extra loop-level code.

Results. 208 tests across three tiers (pure-unit, live-Ray with a stub simulator, and real hardware) all pass. Real-hardware acceptance tests pass for a 1x1 Ariane build and run (`hello_world.c`, compiled and executed by the adapter itself) and for two real, independent checkouts building in parallel: 176s versus 324s serial. One acceptance test, 2x2 Ariane on a real GCP worker, remains blocked on infrastructure outside this project’s own code (§8).

4. Phase 2: The MACE Loop

Pipeline. Figure 1 shows the full loop. A planning agent turns a `MaceSpec` (core, target mesh, gate workloads, objective, budget) into a task DAG. Workers dispatch dependency-satisfied tasks in parallel through Ray, each pinned to its own checkout. A deterministic integrator, with no LLM in this step, applies accepted diffs to the canonical tree in dependency order, then runs every gate workload’s build and run. The gate is binary: every workload’s *verdict*, parsed from real simulator output, must read `pass`. On any failure, a failure-analysis agent (restricted to read-only tools: `grep`, `status`, `collect`) diagnoses the transcript and proposes a fix. The planner replans around it.

Gate workloads found a real bug. Writing the gate workloads (barrier-and-atomic-counter, producer/consumer, hart-id scatter/gather) surfaced a memory-visibility bug in this RTL/toolchain combination. A plain `volatile` read of a shared array element was not reliably visible from the same hart shortly after the write. An atomic fetch-and-add-zero read of the same value always was. Every gate workload now reads shared values through this `atomic_read()` helper. This is a workload-development finding, not a claim about the loop’s own recovery behavior.

Enforcement, not just measurement. MACE checks all three `Budget` dimensions, iteration count, USD, and wall-clock, before each iteration starts. It does not just log them afterward. We verify gate-workload checksums at run start,

before any LLM call or checkout access, so a task cannot tamper with its own grading.

Replay, scoped honestly. MACE cache-tags LLM, build, and run decisions and re-serves them at no extra token or compute cost. We read CHIA’s own cache and bypass implementation directly and confirmed this replays *decisions* (an LLM’s response text, a build or run’s pass/fail artifact), not the literal source edits an agent made through its tools. Those stay git’s job. This is a real, useful scope, not a shortcoming.

5. A Third Core: PicoRV32

We were asked whether MACE could take arbitrary core RTL and a specification and integrate it into OpenPiton’s fabric automatically. We investigated before writing code and found this framed the problem wrong: OpenPiton has no reusable core-to-NoC bridge. Each core needs its own L15 adapter. Even Ariane’s own AXI4 output goes unused, since a hand-written encoder taps CVA6’s internal pre-AXI cache-miss stream instead. The real dividing line is not “does the core speak a standard bus” but “does the core have its own coherent cache.” Ariane’s situation needs new coherence-adapter RTL, which was outside our remaining time. PicoRV32’s situation is different: a complete, upstream, TODO-free adapter (`pico_l15_transducer.v` and friends) already exists in OpenPiton’s own tile generation logic. Nobody had ever run it under a simulator. OpenPiton’s own CI builds it under Verilator but its run job is commented out, targets a nonexistent stage, and even when live pointed at a different simulator.

Toolchain reuse, confirmed not assumed. The RV32IMA/ILP32 multilib already present in our installed `riscv64-unknown-elf-gcc` covers PicoRV32 with no new toolchain install. We confirmed this with a real trial assemble-and-link of the target diagnostic, not just by checking multilib metadata.

Results. The adapter extension (a new `Literal` entry, two `sims` flags, mirrored tests) landed with full unit coverage and no regressions to the existing `ariane` and `sparc` paths. The build passes in 57 seconds, faster than Ariane’s since pico’s build skips the bootrom/device-tree chain. The initial run reached verdict `maxcycles`.

The compiled binary’s symbol table and entry point matched the run’s own configured trap addresses (`objdump -t/-f` against `symbol.tbl`), which ruled out a bad build or toolchain mismatch. Its transcript matched a known-good passing-Ariane fixture line for line through the boot/reset/IOB-handshake trace. The fixture’s next line, `Hit Good trap`, never appeared for pico. This located the

divergence to the point where the core itself must reach its own trap address, not the boot code or the binary.

We chased this one to a waveform-level root cause, unlike the still-open 2x2 Ariane hang (§8). We found and fixed three real RTL/testbench bugs: PicoRV32’s own `resethn/booted` self-boot gate, the manycore monitor’s `active_thread` tracking for pico’s tile, and a real-silicon BIST self-clear race that discarded pico’s first memory writes. PicoRV32 now **passes** (Simulation -> PASS (HIT GOOD TRAP)), the first pass for this core under any simulator. MACE told “builds” apart from “runs correctly” instead of reporting a false pass, and that distinction let us find and fix the real bug instead of stopping at a shrug.

6. Beyond the Proposal: Interface, Coverage, Portability

MACE adds three capabilities beyond the original proposal. We proved each one against real hardware or a live backend, never a mock.

The `mace` command starts an interactive shell modeled on Yosys and OpenROAD: `read_verilog`, `top_module`, `read_spec`, `set_core`, then `run`. This is the real interface a user drives MACE through, not just an API. When `top_module` names a core `chia_openpiton` does not support, `run` immediately gives the same assessment §5 needed real investigation to reach, instead of attempting a fake integration.

Verilator code coverage (`run -coverage`) is a second verification axis beyond pass/fail. It needed a real fix, not just a flag: OpenPiton’s own hand-written testbench never called Verilator’s coverage-write API, so a clean run produced no data until we patched it. A real run (1x1 Ariane, `barrier_atomic.c`) reports 36.00% (8787/24311), measured on real hardware.

MACE’s loop also runs end to end, unmodified, against Google’s Vertex Gemini backend and OpenCode. We confirmed this with a full real passing run. Vertex authenticates through Application Default Credentials, not a literal key, and we had to find which model was actually available (`gemini-2.0-*` returns 404; `gemini-2.5-flash` works).

7. Evaluation

We compare three approaches. (a) is manual mesh scaling with a time log. (b) is a single LLM prompt for a configuration, applied once with no tools, verification, or retry. (c) is the full MACE loop. Every result below comes from a real hardware run. This paper contains no simulated numbers.

Three cores, two baselines. We ran (b) and (c) against the same objective, verify `barrier_atomic.c` on a 1x1 mesh, for all three supported cores, not just ariane, with the same LLM backend (Gemini 2.5 Flash) throughout (Figure 2, left).

(b) fails all three cores, each in a different way. On ariane, the one-shot config built but failed real verification:

a wrong guess. On sparc, it built and reached a run attempt that failed silently, with no verdict at all. On pico, `PitonConfig`’s own construction-time validation rejected the LLM’s config (`l15_size=0`) before any hardware ran; (b) has no retry path for a malformed response.

(c) passes ariane: 4/4 tasks, 741.7s, about 4x (b)’s cost, not “for free” as an earlier, buggy measurement suggested. A second, different ariane gate workload, `producer_consumer.c`, also passed (2118.02s, 2 tasks), real coverage beyond one program. On sparc and pico, (c) does not pass either. It exhausts its retry budget instead. Its own triage and post-mortem, not us reading logs by hand, told the two cases apart: sparc’s 3 replan attempts could not resolve a missing `util.h` include path in the gate program’s build (classified `fixable_config`, an LLM diagnosis we have not independently checked); pico’s 3 attempts all deadlocked (`maxcycles`) because `barrier_atomic.c`’s barrier logic needs more than one participant, something a 1x1 mesh cannot provide (classified `likely_hardware_limitation`, more precisely a workload/mesh mismatch, not a return of the pico RTL bugs §5 already fixed).

The loop passes mesh shapes manual scaling could not. Baseline (a) got stuck at 2x2 (build passes, run hangs) and never reached 4x4; §8 covers the two real causes. The full loop passed both, on real hardware (Figure 2, right).

Phase 2’s own fan-out (two real tasks, two checkouts, through baseline (c)’s own path) measured 30s parallel versus 57s serial, matching Phase 1’s result.

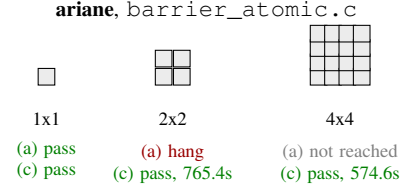
One gap remains honest: we proved the detect-failure, diagnose, replan, pass cycle only at the mechanism level (a stateful stub that fails once, then passes). It never fired across the passing runs above. This includes a forced extreme first config (a 128-byte, direct-mapped L1, confirmed actually used) that we and the planner both expected to fail. It passed instead. This is a real finding about this RTL’s coherence robustness to cache-capacity extremes, not a test-design failure, and we report it as such instead of letting repeated passes claim more than they show.

8. Limitations and Honest Findings

A launch-configuration gap we first misdiagnosed as a CHIA scheduler bug. A task whose resource request could only be satisfied by a tailnet-relayed GCP worker never ran, even though that worker’s own registration and heartbeat to the cluster’s GCS worked the whole time. We ruled out a worker-startup failure (the node stayed Alive with correct resources) and scheduler slowness (a six-minute state-dump showed zero pending start requests). We first concluded this was a real gap in CHIA’s own scheduling and relay code and filed it upstream (`ucb-bar/chia#72`). A CHIA maintainer’s reply found the real cause: CHIA sets three proxy environment variables on each node during `chia up`. A job submitted through `chia job submit` inherits them; a driver launched manually through `python driver.py`, which every real run here uses, does not, unless the launching shell sets them itself. We verified this directly:

Core	(b) one-shot LLM	(c) full loop
ariane	fail, 186.5s	pass, 741.7s
sparc	fail, 766.0s	budget_exceeded, 966.3s
pico	fail (unparseable), 32.6s	budget_exceeded, 1250.5s

Figure 2. **Left:** all three cores, same 1x1 objective. (b) never passes. On sparc/pico, (c) exhausts its retry budget instead of reporting a false pass. Only ariane reaches a real hardware **pass** under either baseline (logs: `runs/baseline_{b,c}_output.log` for ariane, `runs/bench_{sparc,pico}_{b,c}.log` for sparc/pico). **Right:** manual mesh scaling (a) vs. (c), ariane only. **Green**=pass, **red**=hang, **gray**=not reached. The loop passes both shapes (a) got stuck on.



exporting those values from the head’s own live raylet took the same task from a six-minute hang to a two-second run. Every fact we found was correct. Our first conclusion was not, and we correct it here instead of leaving it standing. With dispatch fixed, 2x2 Ariane on GCP still has not reached a verdict, likely the same memory-pressure class as below.

A later, independent dispatch attempt on the same topology reproduced the stuck symptom, with the environment fix already confirmed in place, which rules out a repeat of the same cause. Both tailnet relays’ logs and a raylet-level `strace` show zero new outbound activity. This points at Ray’s own scheduling code under a proxied topology, not a CHIA gap. We filed this as a follow-up and it is still open.

A 2x2 Ariane hang, likely a real RTL gap. The 2x2 Ariane mesh shows the same signature PicoRV32’s investigation started from (§5): a correct boot/reset/IOB handshake, then the core never reaches its own trap address, despite a correct binary. OpenPiton’s own CI has no upstream Verilator record for 2x2 Ariane at any tile count besides 1x1 and 4x4. We read this as a likely, previously undiscovered RTL gap, not an adapter bug, since the same code drives the passing cases. Unlike PicoRV32’s identical symptom,

we have not chased this one to a waveform-level root cause yet.

4x4 memory pressure, two causes. The one upstream-validated 4x4 shape hit two real memory-pressure failures. First, a real Ray-reported out-of-memory condition (22.33GB/23.47GB used) from unconstrained parallel compilation; we fixed this with `MAKEFLAGS`. Second, Verilator’s own build step passes an explicit, unlimited `-j` on its command line, which silently overrides that fix. We confirmed both directly through `/proc`. A later, solo full-loop attempt passed instead (Figure 2, right, 574.6s), consistent with the OOM coming specifically from concurrent compilation this run avoided.

9. Conclusion

MACE shows that an agentic loop can plan, build, and verify real multicore hardware, gated by real RTL simulation instead of an agent’s word. It fixed real environment bugs, added a third core, and passed mesh shapes never reached before. Precision, not a single pass, is the point.